

ΓΡΑΦΙΚΑ ΥΠΟΛΟΓΙΣΤΩΝ ΚΑΙ
ΣΥΣΤΗΜΑΤΑ ΑΛΛΗΛΕΠΙΔΡΑΣΗΣ

1Α ΠΡΟΓΡΑΜΜΑΤΙΣΤΙΚΗ ΕΡΓΑΣΙΑ
ΓΙΑ ΤΟ ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2024-
2025

ΟΜΑΔΑ 5380-5226

ΙΩΑΝΝΗΣ ΦΙΛΛΗΣ, ΑΜ:5380

ΚΩΝΣΤΑΝΤΙΝΟΣ ΖΩΗΣ, ΑΜ:5226

ΤΕΛΙΚΗ ΑΝΑΦΟΡΑ

Νοέμβριος 2024

1 ΕΙΣΑΓΩΓΗ

Η παρούσα αναφορά συγγράφεται στα πλαίσια της πρώτης εργαστηριακής/προγραμματιστικής άσκησης (1Α) του μαθήματος Γραφικά Υπολογιστών και Συστήματα Αλληλεπίδρασης από τους φοιτητές Γιάννη Φύλλη (ΑΜ:5380) και Κωνσταντίνο Ζώη (ΑΜ: 5226).

Σκοπός της άσκησης είναι να εξοικειωθούμε με την χρήση βασικών βιβλιοθηκών της OpenGL. Έτσι, το ζητούμενο είναι να δημιουργήσουμε ένα παιχνίδι, όπου θα ανοίγει σε νέο παράθυρο και θα ζωγραφίζεται ένας λαβύρινθος, μαζί με έναν τετράγωνο χαρακτήρα, ο οποίος θα έχει την δυνατότητα να κινείται μέσα στον λαβύρινθο.

2 ΕΡΩΤΗΜΑ Ι: ΑΝΟΙΓΜΑ ΠΑΡΑΘΥΡΟΥ ΚΑΙ ΤΙΤΛΟΣ

Στο πρώτο ερώτημα αυτό που καλούμαστε να κάνουμε είναι να δημιουργήσουμε το παράθυρο με διαστάσεις 750x750, να θέσουμε τον τίτλο του παραθύρου **Άσκηση 1Α-2024**, να θέσουμε το background του παραθύρου μαύρο και να τερματίζει με το πάτημα του κουμπιού **Q**.

Έτσι, πάμε στον κώδικά μας και βρίσκουμε την εντολή **glfwCreateWindow()** και αλλάζουμε την παράμετρο που αναφέρεται στις διαστάσεις του παραθύρου σε 750x750.

```
// Open a window and create its OpenGL context
window = glfwCreateWindow(750, 750, u8"Άσκηση 1Α - 2024", NULL, NULL);
```

Εικόνα 2.1: Εντολή δημιουργίας παραθύρου

Για τον τίτλο του παραθύρου θέτουμε στην παράμετρο για το όνομα, **u8"Άσκηση 1Α - 2024"**. Χρειάζεται και το u8 μπροστά, διότι χωρίς αυτό δεν μπορεί να αναγνωρίσει τους ελληνικούς χαρακτήρες. Για να αλλάξουμε το χρώμα του background σε μαύρο, βρίσκουμε την εντολή **glClearColor()** και θέτουμε τις παραμέτρους σε 0.0f (επειδή είναι rgb).

```
// black background
glClearColor(0.0f, 0.0f, 0.0f, 0.0f);
```

Εικόνα 2.2: Εντολή αλλαγής χρώματος background

Για το τελευταίο ζητούμενο αυτού του ερωτήματος, θα βρούμε στον κώδικα το κύριο loop και θα τροποποιήσουμε την συνθήκη του, ώστε ο έλεγχος για το αν θα κλείσει το παράθυρο να γίνεται για το κουμπί **Q**.

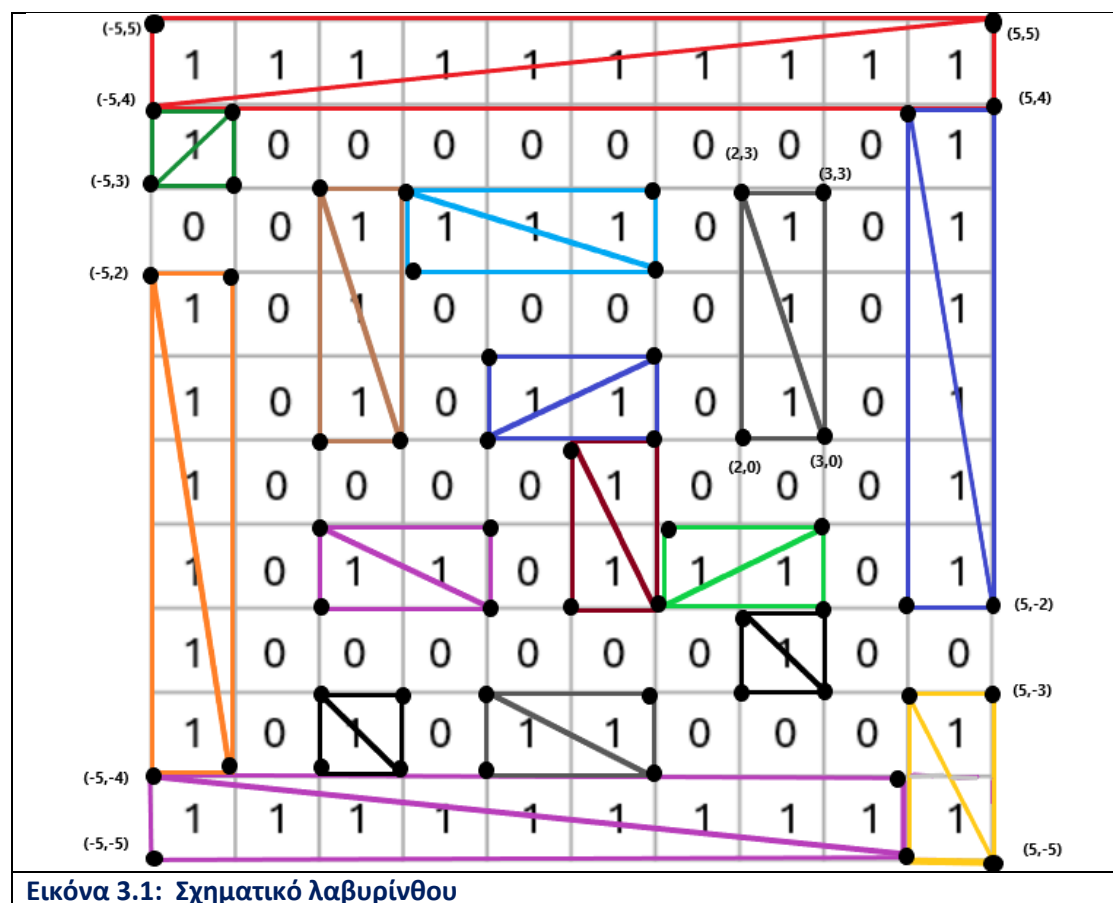
```
//check if q is pressed
while (glfwGetKey(window, GLFW_KEY_Q) != GLFW_PRESS && glfwWindowShouldClose(window) == 0);
```

Εικόνα 2.3: Συνθήκη για έλεγχο πατήματος του Q

3 ΕΡΩΤΗΜΑ ΙΙ: ΛΑΒΥΡΙΝΘΟΣ

Αμέσως μετά, αυτό που καλούμαστε να κάνουμε είναι να δημιουργήσουμε τον λαβύρινθο μας. Για να το κάνουμε αυτό χρειαζόμαστε τετράγωνα. Αυτά, μπορούν να δημιουργηθούν πολύ εύκολα, με την χρήση δύο (2) ορθογώνιων τριγώνων. Για να φτιάξουμε τον λαβύρινθο και να σκεφτούμε πως ακριβώς θα είναι, κάναμε χρήση της εικόνας 2 από την εκφώνηση, όπου μας δίνονταν η αναπαράσταση του λαβυρίνθου η οποία όπου είχε 1 θα περιέχει τον τοίχο και όπου είχε 0 τον διάδρομο.

Στην ίδια εικόνα, μας δίνεται και το μέγεθος του λαβυρίνθου, ο οποίος είναι 10x10 τετραγωνάκια, εφόσον και αυτά έχουν μέγεθος 1. Η πρώτη μας σκέψη ήταν να βάλουμε 2 τριγωνάκια για κάθε «κουτάκι», ώστε να φτιάξουμε πολλά μικρά τετράγωνα, ωστόσο μετά σκεφτήκαμε κάτι αρκετά καλύτερο. Στα σημεία του λαβυρίνθου όπου έχουμε πολλά τετράγωνα το ένα δίπλα στο άλλο, μπορούμε να το αναπαραστήσουμε ως ένα μεγάλο παραλληλόγραμμο, με την χρήση δύο μεγαλύτερων τριγώνων. Παρακάτω ακολουθεί η εικόνα με τις σχεδιάσεις των τοίχων, με την χρήση των τριγώνων για να γίνει περισσότερο κατανοητό. Έχουμε σημειώσει και τις γωνίες που έχουμε χρησιμοποιήσει.



Εικόνα 3.1: Σχηματικό λαβυρίνθου

Οι συντεταγμένες για το κάθε τετραγωνάκι προκύπτουν πολύ εύκολα, αφού το μέγεθος τους είναι 1, για κάθε τετραγωνάκι θα μεταβάλλουμε είτε το x κατά $+1$ ή -1 είτε το y κατά $+1$ ή -1 . Τα όρια του λαβυρίνθου είναι οι συντεταγμένες $(-5,5)$, $(5,5)$, $(5,-5)$, $(-5,-5)$. Έτσι προκύπτουν και οι υπόλοιπες συντεταγμένες των γωνιών του κάθε τετραγώνου στο

σχήμα, όπου εμείς επιλέξαμε ενδεικτικά να γράψουμε κάποιες συντεταγμένες γωνιών, ώστε να παραμείνει κατανοητό το σχήμα και να μην το υπεργεμίσουμε με πληροφορίες.

Αφού τελειώσαμε με αυτό, στον κώδικά μας δημιουργήσαμε το **maze_buffer** στο οποίο αποθηκεύουμε τις συντεταγμένες των κορυφών των τριγώνων μας στην μορφή (x,y,z) όπου το z είναι μονίμως ίσο με 0.

```
static const GLfloat maze_buffer[] = {  
    // (-5,5) (-5,4) (5,5) (5,4) block  
    -5.0f, 5.0f, 0.0f,  
    -5.0f, 4.0f, 0.0f,  
    5.0f, 5.0f, 0.0f,  
    -5.0f, 4.0f, 0.0f,  
    5.0f, 5.0f, 0.0f,  
    5.0f, 4.0f, 0.0f,  
    // (-5,4) (-5,3) (-4,4) (-4,3) block  
    -5.0f, 4.0f, 0.0f,  
    -5.0f, 3.0f, 0.0f,  
    -4.0f, 4.0f, 0.0f,  
    -5.0f, 3.0f, 0.0f,  
    -4.0f, 4.0f, 0.0f,  
    -4.0f, 3.0f, 0.0f,  
    // (4,4) (5,4) (4,-2) (5,-2) block  
    4.0f, 4.0f, 0.0f,  
    5.0f, 4.0f, 0.0f,  
    5.0f, -2.0f, 0.0f,  
    4.0f, 4.0f, 0.0f,  
    4.0f, -2.0f, 0.0f,  
    5.0f, -2.0f, 0.0f,  
    // (4,-3) (4,-5) (5,-3) (5,-5) block  
    4.0f, -3.0f, 0.0f,  
    5.0f, -3.0f, 0.0f,  
    5.0f, -5.0f, 0.0f,  
    4.0f, -3.0f, 0.0f,  
    4.0f, -5.0f, 0.0f,  
    5.0f, -5.0f, 0.0f,  
};
```

Εικόνα 3.2: Ένα κομμάτι από το maze_buffer

Έπειτα κάνουμε τις απαραίτητες ρυθμίσεις για το VAO και VBO.

```
GLuint vertexbuffer;  
glGenBuffers(1, &vertexbuffer);  
glBindBuffer(GL_ARRAY_BUFFER, vertexbuffer);  
glBufferData(GL_ARRAY_BUFFER, sizeof(maze_buffer), maze_buffer, GL_STATIC_DRAW);
```

Εικόνα 3.3: Ρυθμίσεις

Εφόσον τελειώσουμε και με αυτό, το επόμενο βήμα είναι να πάμε στον κύριο βρόγχο και να θέσουμε τα attributes. Βάζουμε λοιπόν να διαβάζει τις συντεταγμένες ανά τρία (x,y,z).

```
glEnableVertexAttribArray(0);  
glBindBuffer(GL_ARRAY_BUFFER, vertexbuffer);  
glVertexAttribPointer(  
    0, // attribute 0, must match the layout in the shader.  
    3, // size  
    GL_FLOAT, // type  
    GL_FALSE, // normalized?  
    0, // stride  
    (void*)0 // array buffer offset  
);  
// Draw the maze  
glDrawArrays(GL_TRIANGLES, 0, 96);  
glDisableVertexAttribArray(0);
```

Εικόνα 3.4: Τελικές ρυθμίσεις και ζωγράφισμα λαβυρίνθου

Έτσι, γράφουμε και την εντολή **glDrawArrays()** με ορίσματα **GL_TRIANGLES**, εφόσον έχουμε τρίγωνα που ζωγραφίζουμε, **0** για να αρχίσει από το index 0 και τέλος **96** εφόσον έχουμε 96 κορυφές που πρέπει να ζωγραφίσει.

Το τελευταίο που έχουμε να κάνουμε είναι να πάμε στο αρχείο **ProjectFragmentShader.fragmentshader** και να τροποποιήσουμε την μεταβλητή `color` ώστε να παίρνει τιμή κατάλληλη για να είναι το χρώμα μπλε (σε rgb).

```
// Output color = blue  
color = vec3(0,0,1);
```

Εικόνα 3.5: Αλλαγή χρώματος τοίχων σε μπλε

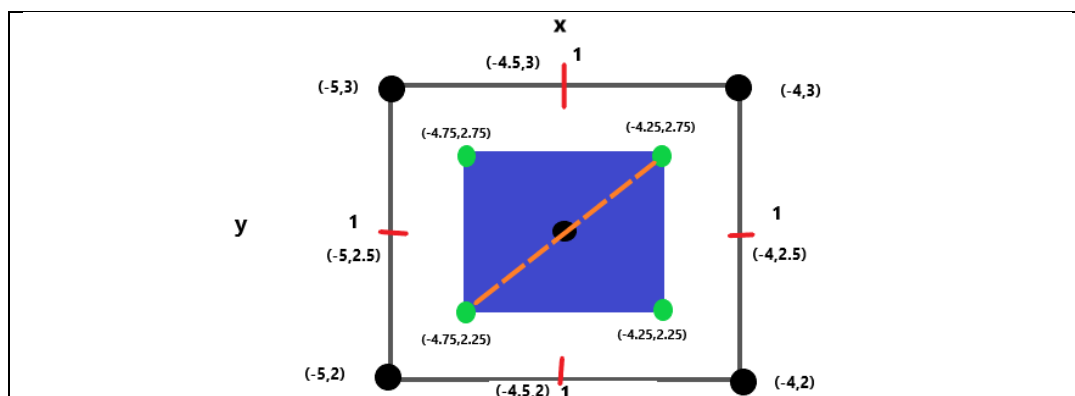
4 ΕΡΩΤΗΜΑ ΙΙΙ: ΣΧΕΔΙΑΣΜΟΣ ΧΑΡΑΚΤΗΡΑ

Στο συγκεκριμένο ερώτημα το ζητούμενο είναι να σχεδιάσουμε τον χαρακτήρα μας, ο οποίος θα κινείται μέσα στον λαβύρινθο. Όπως μας δίνεται στην εκφώνηση, το μήκος της πλευράς του χαρακτήρα είναι 0.5 και το κέντρο του είναι το κέντρο του τετραγώνου στο οποίο βρίσκεται.

Αυτό ουσιαστικά σημαίνει, πως αν χωρίζαμε το τετραγωνάκι (χαρακτήρα) στην μέση κατακόρυφα, τότε η οι αριστερές γωνίες του θα ήταν 0.25 αριστερά από το κέντρο του ως προς το x. Ομοίως οι δεξιές γωνίες του θα βρίσκονται 0.25 δεξιά από το κέντρο του ως προς το x.

Αν κάναμε την ίδια διαδικασία, αλλά τώρα χωρίζαμε τον χαρακτήρα οριζόντια, θα προέκυπτε πως οι πάνω γωνίες του βρίσκονται πάνω από το κέντρο του κατά 0.25, ενώ οι κάτω γωνίες του βρίσκονται κάτω από το κέντρο του κατά 0.25.

Αυτό ακριβώς παρουσιάζεται στην εικόνα 4.1, όπου έχουμε το μεγάλο τετραγωνάκι το οποίο είναι το τετράγωνο στο οποίο θα αρχίζει ο χαρακτήρας μας. Το μπλε τετράγωνο είναι ο χαρακτήρας A. Έχουμε σημειώσει τις συντεταγμένες τόσο του μεγάλου τετραγώνου όσο και των γωνιών του χαρακτήρα που προκύπτουν. Με την πορτοκαλί διακεκομμένη γραμμή γίνονται εμφανή τα δύο τρίγωνα που σχηματίζουν τον χαρακτήρα μας. Οι κόκκινες γραμμές βοηθούν στο να αντιληφθούμε το κέντρο του χαρακτήρα. Οπότε προκύπτει, πως οι γωνίες που απαρτίζουν τον χαρακτήρα μας έχουν συντεταγμένες $(-4.75, 2.75)$, $(-4.25, 2.75)$, $(-4.25, 2.25)$, $(-4.75, 2.25)$. Φυσικά, αυτές είναι οι αρχικές αφού θα αλλάζουν καθώς κινείται ο A.



Εικόνα 4.1: Χαρακτήρας A

Έτσι λοιπόν, για να προχωρήσουμε στο προγραμματιστικό κομμάτι, θα αποθηκεύσουμε τις (αρχικές) συντεταγμένες του A στην `character_buffer()`.

```
static GLfloat character_buffer[] = {  
    // (-4.75, 2.25) (-4.25, 2.25) (-4.75, 2.75) (-4.25, 2.75) block  
    -4.75f, 2.75f, 0.0f, //xmin,ymax point  
    -4.75f, 2.25f, 0.0f,  
    -4.25f, 2.75f, 0.0f,  
    -4.75f, 2.25f, 0.0f,  
    -4.25f, 2.75f, 0.0f,  
    -4.25f, 2.25f, 0.0f  
};
```

Εικόνα 4.2: συντεταγμένες του A

Έπειτα, γράφουμε την εντολή που φαίνεται στην εικόνα 4.3 και συνεχίζουμε μέσα στον βρόγχο.

```
GLuint characterbuffer;  
glGenBuffers(1, &characterbuffer);
```

Εικόνα 4.3: Εντολή για τον χαρακτήρα

Μέσα στον βρόγχο τώρα, θα φτιάξουμε πάλι τις ρυθμίσεις για τον χαρακτήρα αυτή την φορά. Οι ρυθμίσεις είναι ίδιες και δημιουργούμε διαφορετικό buffer για τον χαρακτήρα, ωστόσο θα πρέπει στην εντολή **glBufferData()** να περάσουμε την παράμετρο **GL_DYNAMIC_DRAW** η οποία χρειάζεται αργότερα για την

```
// character  
glEnableVertexAttribArray(0);  
glBindBuffer(GL_ARRAY_BUFFER, characterbuffer);  
glBufferData(GL_ARRAY_BUFFER, sizeof(character_buffer), character_buffer, GL_DYNAMIC_DRAW);  
  
glVertexAttribPointer(  
    0,           // attribute 0, must match the layout in the shader.  
    3,           // size  
    GL_FLOAT,    // type  
    GL_FALSE,    // normalized?  
    0,           // stride  
    (void*)0     // array buffer offset  
);  
//draw character  
glDrawArrays(GL_TRIANGLES, 0, 6);  
glDisableVertexAttribArray(0);
```

Εικόνα 4.4: Εντολές για τον χαρακτήρα μέσα στον βρόγχο

κίνηση του χαρακτήρα. Τέλος, γράφουμε πάλι την εντολή **glDrawArrays()** και αυτή την φορά ως όρισμα για τις γωνίες βάζουμε **6**. Φυσικά, στο τέλος διαγράφουμε τους buffers του λαβυρίνθου και του χαρακτήρα.

```
// Cleanup VBO  
glDeleteBuffers(1, &vertexbuffer);  
glDeleteBuffers(1, &characterbuffer);  
glDeleteVertexArrays(1, &VertexArrayID);  
glDeleteProgram(programID);
```

Εικόνα 4.5: Διαγραφή των buffers

5 ΕΡΩΤΗΜΑ IV: ΚΙΝΗΣΗ ΚΑΙ ΛΑΒΥΡΙΝΘΟΣ

Το συγκεκριμένο ερώτημα ήταν αρκετά μεγάλο, οπότε φροντίσαμε να το χωρίσουμε σε υποερωτήματα/υποενότητες.

5.1 ΚΙΝΗΣΗ ΧΑΡΑΚΤΗΡΑ

Για την κίνηση του χαρακτήρα, υλοποιήσαμε την συνάρτηση *moveSquare()*. Το πρώτο της όρισμα είναι ένας πίνακας **coord**, δηλαδή ο πίνακας με τα vertices του αντικειμένου που θέλουμε να μετακινήσουμε. Ο πίνακας **map** αποθηκεύει τον χάρτη του λαβυρίνθου και είναι βοηθητικός για την σύγκρουση στους τοίχους. Αυτό θα το αναλύσουμε αργότερα. Τέλος, η μεταβλητή **move** αναπαριστά τον κωδικό της κίνησης.

Το πρώτο πράγμα που κάνουμε είναι να αντιγράψουμε τον πίνακα **coord** σε έναν προσωρινό **temp** ώστε στη συνέχεια να ελέγχουμε αν πέφτουμε πάνω σε τοίχο με τον προσωρινό πίνακα, για να μην χαλάσουμε τα vertices του χαρακτήρα μας. Για αυτό σε κάθε περίπτωση αλλάζουμε αρχικά τον πίνακα **temp** και καλούμε την συνάρτηση *checkMove()*. Αν η κίνηση είναι έγκυρη, αλλάζουμε τον κανονικό πίνακα. Η *checkMove()* θα περιγραφεί με λεπτομέρεια παρακάτω.


```
139
140 void moveSquare(GLfloat* coord, GLfloat map[10][10], int move) {
141
142     GLfloat temp[18];
143     for (int i = 0; i < 18; i++) {
144         temp[i] = coord[i];
145     }
146     if (move == 1) { //move right
147         for (int i = 0; i < 18; i = i + 3) {
148             temp[i] = temp[i] + 1.0f;
149         }
150         if (checkMove(map, temp) != false) {
151             for (int i = 0; i < 18; i = i + 3) {
152                 coord[i] = coord[i] + 1.0f;
153             }
154         }
155     }
156     else if (move == 2) { // move left
157         for (int i = 0; i < 18; i = i + 3) {
158             temp[i] = temp[i] - 1.0f;
159         }
160         if (checkMove(map, temp) != false) {
161             for (int i = 0; i < 18; i = i + 3) {
162                 coord[i] = coord[i] - 1.0f;
163             }
164         }
165     }
166     else if (move == 3) { // move up
167         for (int i = 1; i < 18; i = i + 3) {
168             temp[i] = temp[i] + 1.0f;
169         }
170         if (checkMove(map, temp) != false) {
171             for (int i = 1; i < 18; i = i + 3) {
172                 coord[i] = coord[i] + 1.0f;
173             }
174         }
175     }
176     else { //move down
177         for (int i = 1; i < 18; i = i + 3) {
178             temp[i] = temp[i] - 1.0f;
179         }
180         if (checkMove(map, temp) != false) {
181             for (int i = 1; i < 18; i = i + 3) {
182                 coord[i] = coord[i] - 1.0f;
183             }
184         }
185     }
186 }
187
```

Εικόνα 5.1: Κώδικας συνάρτησης moveSquare()

Αν ο κωδικός move είναι ίσος με 1 ή 2 τότε κινούμαστε δεξιά ή αριστερά αντίστοιχα. Σε αυτή την περίπτωση, πρέπει να προσθέσουμε 1 (δεξιά) ή να αφαιρέσουμε 1 (αριστερά) στις x συντεταγμένες του χαρακτήρα. Ξέρουμε ότι η x συντεταγμένη είναι η πρώτη για κάθε σημείο στον πίνακα **character_buffer**, έτσι πρέπει να διατρέξουμε τον πίνακα ξεκινώντας από το 0 με βήμα 3 κάθε φορά. Αν ο κωδικός είναι ίσος με 3, τότε κινούμαστε πάνω ενώ αν είναι οτιδήποτε άλλο κινούμαστε κάτω. Εδώ πρέπει να αλλάξουν οι y συντεταγμένες κατά +1 (πάνω) ή -1 (κάτω). Αφού το y αποθηκεύεται δεύτερο για κάθε σημείο, για να διατρέξουμε τον πίνακα ξεκινάμε από το 1 με βήμα 3 ξανά. Σε κάθε επανάληψη ορίζουμε το i ίσο με 18, αφού ο χαρακτήρας αποτελείται από 6 σημεία με 3 συντεταγμένες το καθένα.

Επομένως, μας μένει να καλέσουμε την συνάρτησή μας μέσα στη main με όρισμα τον **character_buffer** που περιέχει τα vertices του χαρακτήρα και με τον κατάλληλο κωδικό ανάλογα με το κουμπί που θα πατηθεί. Σε αυτό το σημείο αντιμετωπίσαμε κάποια προβλήματα, όπως το ότι ο χαρακτήρας κινούνταν πολύ γρήγορα στο λαβύρινθο, ανάλογα με την πίεση στο πάτημα του πλήκτρου και είχε συνεχόμενη κίνηση αν κρατούσαμε το πλήκτρο πατημένο. Ύστερα από διευκρινίσεις στο φόρουμ του μαθήματος, καταλάβαμε ότι πρέπει να εφαρμόσουμε κάποια καθυστέρηση στο input. Ωστόσο ακόμα και έτσι (εφαρμόζοντας μόνο αυτό δηλαδή) ορισμένες φορές έκανε και πάλι κάποια έξτρα βήματα.

Για αυτό το σκοπό, ορίσαμε τις boolean μεταβλητές **key_pressed** για κάθε πλήκτρο. Έτσι, ελέγχουμε ότι πατιέται ένα πλήκτρο με το **GLFW_PRESS** και αν δεν είναι ήδη πατημένο καλούμε την **moveSquare()**. Ενδιάμεσα από αυτό το κάλεσμα, κάνουμε sleep για κάποιο ενδεικτικό διάστημα ώστε να σιγουρευτούμε ότι θα διατηρηθεί το delay και έπειτα θέτουμε την **key_pressed** σε true. Αφού δουλεύουμε μέσα σε ένα βρόγχο που τρέχει συνεχώς, δεν θέλουμε να υπάρχει επικάλυψη των input και για αυτό ελέγχουμε όταν αφεθεί ένα πλήκτρο με το **GLFW_RELEASE**. Μετά φυσικά θέτουμε την boolean μεταβλητή false.

```
566
567     if (glfwGetKey(window, GLFW_KEY_L) == GLFW_PRESS && !keyLpressed) { // right move
568         Sleep(50);
569         moveSquare(character_buffer, map, 1);
570         Sleep(50);
571         keyLpressed = true;
572     }
573
574     else if (glfwGetKey(window, GLFW_KEY_L) == GLFW_RELEASE) {
575         keyLpressed = false;
576     }
577
578     if (glfwGetKey(window, GLFW_KEY_J) == GLFW_PRESS && !keyJpressed) { // left move
579         Sleep(50);
580         moveSquare(character_buffer, map, 2);
581         Sleep(50);
582         keyJpressed = true;
583     }
584     else if (glfwGetKey(window, GLFW_KEY_J) == GLFW_RELEASE) {
585         keyJpressed = false;
586     }
587
588     if (glfwGetKey(window, GLFW_KEY_I) == GLFW_PRESS && !keyIpressed) { // up move
589         Sleep(50);
590         moveSquare(character_buffer, map, 3);
591         Sleep(50);
592         keyIpressed = true;
593     }
594     else if (glfwGetKey(window, GLFW_KEY_I) == GLFW_RELEASE) {
595         keyIpressed = false;
596     }
597
598     if (glfwGetKey(window, GLFW_KEY_K) == GLFW_PRESS && !keyKpressed) { // down move
599         Sleep(50);
600         moveSquare(character_buffer, map, 4);
601         Sleep(50);
602         keyKpressed = true;
603     }
604     else if (glfwGetKey(window, GLFW_KEY_K) == GLFW_RELEASE) {
605         keyKpressed = false;
606     }
```

Εικόνα 5.2: Κλήση της moveSquare() στην main

5.2 ΣΥΓΚΡΟΥΣΗ ΧΑΡΑΚΤΗΡΑ ΣΕ ΤΟΙΧΟ

Το πρόβλημα που αντιμετωπίσαμε στην σύγκρουση του χαρακτήρα με τους τοίχους ήταν ότι δεν είχαμε κάποια δομή με τα μπλοκ του λαβυρίνθου για να γνωρίζουμε αν σε κάθε θέση ο χαρακτήρας έχει πέσει πάνω τους. Στον **maze_buffer** έχουμε ορίσει διάφορα τρίγωνα και αν τα μπλοκ είναι συνεχόμενα, οι συντεταγμένες που έχουμε για τα τρίγωνα μπορεί να επικαλύπτουν πολλά τετράγωνα μαζί. Για να το αντιμετωπίσουμε αυτό ορίσαμε στη **main** έναν δισδιάστατο πίνακα **map**. Αυτός ο πίνακας είναι ουσιαστικά ο χάρτης του λαβυρίνθου όπου το 1 σημαίνει ότι υπάρχει τοίχος. Για αυτό, εμπνευστήκαμε από την εικόνα 2 στην εκφώνηση της άσκησης που απεικονίζει μια αναπαράσταση του λαβυρίνθου. Ο **map** λοιπόν είναι ακριβώς αυτός ο πίνακας της εικόνας.

Επομένως στην **main** ορίζουμε **GLfloat map[10][10]** και καλούμε **createMap(map)**. Αυτή η συνάρτηση γεμίζει με μηδενικά όλο τον πίνακα και προσθέτει τους τοίχους, θέτει δηλαδή 1 σε όποιο σημείο χρειάζεται.

```
213
214 void createMap(GLfloat map[10][10]) {
215     for (int i = 0; i < 10; i++) {
216         for (int j = 0; j < 10; j++) {
217             map[i][j] = 0.0f;
218         }
219     }
220     for (int k = 0; k < 10; k++) {
221         map[0][k] = 1.0f;
222     }
223     map[1][0] = 1.0f;
224     for (int l = 1; l < 7; l++) {
225         map[l][9] = 1.0f;
226     }
227     for (int l = 3; l < 9; l++) {
228         map[l][0] = 1.0f;
229     }
230     for (int l = 0; l < 10; l++) {
231         map[9][l] = 1.0f;
232     }
233
234     map[2][2] = 1.0f, map[3][2] = 1.0f, map[4][2] = 1.0f;
235     map[2][3] = 1.0f, map[2][4] = 1.0f, map[2][5] = 1.0f;
236     map[2][7] = 1.0f, map[3][7] = 1.0f, map[4][7] = 1.0f;
237     map[4][4] = 1.0f, map[4][5] = 1.0f;
238     map[6][2] = 1.0f, map[6][3] = 1.0f;
239     map[5][5] = 1.0f, map[6][5] = 1.0f;
240     map[6][6] = 1.0f, map[6][7] = 1.0f;
241     map[8][4] = 1.0f, map[8][5] = 1.0f;
242     map[8][2] = 1.0f;
243     map[7][7] = 1.0f;
244     map[8][9] = 1.0f;
245
246 }
```

Εικόνα 5.3: Η συνάρτηση **createMap()**

Σε αυτό το σημείο θα περάσουμε στην συνάρτηση **checkMove()**. Η συνάρτηση αυτή, για να ελέγχει αν ο χαρακτήρας έχει πέσει πάνω σε τοίχο, παίρνει ως όρισμα τον χάρτη του

λαβυρίνθου και τον πίνακα του χαρακτήρα. Η ιδέα είναι να αντιστοιχεί τη θέση του χαρακτήρα με αυτή του χάρτη και αν είναι 1 να επιστρέφει false. Αρχικά ελέγχει αν το x του παίκτη είναι μεγαλύτερο από 5 ή μικρότερο από -5. Αν ισχύει αυτό, σημαίνει ότι ο παίκτης έχει φύγει από την είσοδο ή την έξοδο και βρίσκεται εκτός λαβυρίνθου.

Το πρόβλημα που έχουμε εδώ όμως είναι ότι το τετράγωνο του χαρακτήρα είναι μικρότερης κλίμακας από αυτά του λαβυρίνθου. Πρέπει λοιπόν να το μεγαλώσουμε σε ολόκληρο τετράγωνο. Αυτό γίνεται με την συνάρτηση **upscalePlayer()**. Αντί να “μεγαλώσουμε” όλα τα σημεία, θα μετατρέψουμε σε μεγαλύτερη κλίμακα μόνο το πάνω αριστερά σημείο (δηλαδή το (x_{min}, y_{max})) και θα δουλέψουμε μόνο με αυτό. Εξάλλου αυτό θα είναι μόνο προσωρινό, δεν θέλουμε να αλλάξουμε πραγματικά τις διαστάσεις του χαρακτήρα. Για να επιτευχθεί αυτό, η συνάρτησή μας έχει όρισμα έναν πίνακα, τον πίνακα με τα vertices του παίκτη δηλαδή κατά την κλήση. Κατά τον ορισμό του **character_buffer** θέσαμε στην πρώτη θέση το σημείο (x_{min}, y_{max}) . Οπότε στη συνάρτηση το **a[0]** είναι το x_{min} , ενώ το **a[1]** είναι το y_{max} . Ανάλογα με τη θέση του τετραγώνου, θα πρέπει να προσθέσουμε ή να αφαιρέσουμε 0,25 στα σημεία του, αφού όπως εξηγήσαμε και πριν κατά τόσο διαφέρει στο μέγεθος από τα μπλοκ του λαβυρίνθου. Ο πίνακας **temp** αποθηκεύει απλά στις δύο θέσεις του το νέο (x_{min}, y_{max}) και αυτός επιστρέφεται στο τέλος.

```
190
191 //upscale player to a full block and store (xmin,ymax)
192 GLfloat* upscalePlayer(GLfloat a[]) {
193     GLfloat temp[2] = { 0.0f, 0.0f };
194     GLfloat x = a[0];
195     GLfloat y = a[1];
196     //x
197     if (a[0] < 0) {
198         temp[0] = x - 0.25f;
199     }
200     else {
201         temp[0] = x + 0.25f;
202     }
203     //y
204     if (a[1] < 0) {
205         temp[1] = y - 0.25f;
206     }
207     else {
208         temp[1] = y + 0.25f;
209     }
210     return temp;
211 }
212
```

Εικόνα 5.4: Η συνάρτηση **upscalePlayer()**

Ας επιστρέψουμε στην **checkMove()**. Καλούμε την **upscalePlayer()** και έτσι αποθηκεύουμε τα x_{min}, y_{max} . Τώρα ξέρουμε το πάνω αριστερά σημείο του χαρακτήρα αν είχε το ίδιο μέγεθος με τους τοίχους του λαβυρίνθου και μας μένει να συγκρίνουμε αυτό το σημείο με το αντίστοιχο στον χάρτη για να δούμε αν εκεί βρίσκεται τοίχος. Αυτό δεν είναι τόσο απλό όμως γιατί ο δισδιάστατος πίνακας **map** έχει θέσεις από 0 έως 9 στις γραμμές και στήλες, αλλά τα σημεία του χαρακτήρα μας τα μετράμε από το -5 έως 5. Πρέπει λοιπόν να βρούμε έναν τρόπο ώστε να μετατρέψουμε τα σημεία από -5 έως 5 σε κλίμακα 0 έως 9, για να αντιστοιχούν επακριβώς στον χάρτη. Αν κάνουμε τις κατάλληλες πράξεις στον πίνακα **temp** (ο οποίος έχει τα x_{min}, y_{max}), μετατρέπουμε τα σημεία στην κλίμακα 0 έως 9 και αποθηκεύουμε το x στο **col** και το y στο **row**. Γίνεται ο έλεγχος αν

αυτή η θέση του χάρτη είναι 1 και αν ναι, επιστρέφεται false. Ανάλογα με το τεταρτημόριο που βρισκόμαστε τα x και τα y μπορεί να είναι θετικά ή αρνητικά, γι' αυτό και πρέπει να αλλάξουν λίγο οι πράξεις.

Αυτό που θέλει προσοχή είναι ότι όταν αναφερόμαστε σε ένα στοιχείο από πίνακα μετράμε πρώτα τις γραμμές και μετά τις στήλες (πχ $a[\text{row}][\text{col}]$). Όμως στο συνειδητό σύστημα συντεταγμένων για τα αντικείμενα μας το x ουσιαστικά μετράει στήλη και το y γραμμή. Επομένως για την γραμμή (**row**) αλλάζουμε το $y_{\max}$ ενώ για την στήλη (**col**) αλλάζουμε το $x_{\min}$.

Για παράδειγμα, αν μετά την **upscalePlayer()** το σημείο μας είναι το (-4,-2) αυτό βρίσκεται στο 3^ο τεταρτημόριο και στον πίνακα του χάρτη αντιστοιχεί στο $\text{map}[7,1]$. Όπως εξηγήσαμε πρέπει να μετατρέψουμε το σημείο αυτό και λόγω της ανάποδης αντιστοιχίας των x, y ουσιαστικά θα πρέπει το -4 να γίνει 1 και το -2 να γίνει 7. Με τις πράξεις που βρίσκονται στην συνάρτηση θα γίνει:

$$\text{row} = -(-2) + 5 = 7 \text{ και } \text{col} = -4 + 5 = 1$$

Τέλος, γίνεται ο έλεγχος αν αυτό το σημείο του χάρτη είναι ίσο με 1. ($\text{map}[7][1] == 1$).

```
247
248 //check if move is valid, compare player block with map block
249 bool checkMove(GLfloat map[10][10], GLfloat* player) {
250
251     //check if player is out of map
252     for (int i = 0; i < 18; i = i + 3) {
253         if (player[i] > 5 || player[i] < -5) {
254             return false;
255         }
256     }
257
258     GLfloat* temp = upscalePlayer(player);
259     int row = 0;
260     int col = 0;
261
262     //transform player coords to map cords
263
264     //2nd quarter
265     if (temp[0] < 0 && temp[1] > 0) {
266         row = -(temp[1] - 5);
267         col = temp[0] + 5;
268         if (map[row][col] == 1.0f) {
269             return false;
270         }
271     }
272     //1st quarter
273     else if (temp[0] > 0 && temp[1] > 0) {
274         row = -temp[1] + 5;
275         col = temp[0] + 5;
276         if (map[row][col] == 1.0f) {
277             return false;
278         }
279     }
280     //4th quarter
281     else if (temp[0] > 0 && temp[1] < 0) {
282         row = -temp[1] + 5;
283         col = temp[0] + 5;
284         if (map[row][col] == 1.0f) {
285             return false;
286         }
287     }
288     //3rd quarter
289     else if (temp[0] < 0 && temp[1] < 0) {
290         row = -temp[1] + 5;
291         col = temp[0] + 5;
292         if (map[row][col] == 1.0f) {
293             return false;
294         }
295     }
296
297     return true;
298 }
```

Εικόνα 5.5: Η συνάρτηση checkMove()

6 ΠΛΗΡΟΦΟΡΙΕΣ ΣΧΕΤΙΚΑ ΜΕ ΤΗΝ ΥΛΟΠΟΙΗΣΗ

Κατά την διάρκεια της άσκησης εργαστήκαμε με τα ακόλουθα:

Υπολογιστής 1

Λειτουργικό Σύστημα	Windows 10
Περιβάλλον/editor	Visual Studio 2022 (x64)

Υπολογιστής 2

Λειτουργικό Σύστημα	Windows 11
Περιβάλλον/editor	Visual Studio 2022 (x64)

Δεν υπάρχουν κάποιες ιδιαιτερότητες στον τρόπο εκτέλεσης ή χρήσης.

7 ΑΞΙΟΛΟΓΗΣΗ ΤΗΣ ΛΕΙΤΟΥΡΓΙΑΣ ΤΗΣ ΟΜΑΔΑΣ

Κατά την διάρκεια της εργασίας η συνεργασία μας ήταν άριστη. Πριν ξεκινήσουμε οποιοδήποτε ερώτημα, συζητάγαμε πως θα μπορούσε να υλοποιηθεί το καθένα, αφού αυτό είναι ουσιαστικά σχεδόν πάντα η κύρια δυσκολία. Αφού καταλήγαμε κάπου, ξεκινούσε η υλοποίηση (το προγραμματιστικό κομμάτι δηλαδή).

Το τελευταίο αυτό κομμάτι, προσπαθήσαμε όσο γίνεται να το ισομοιράσουμε. Η συγγραφή της αναφοράς έγινε με όμοιο τρόπο.

8 ΠΗΓΕΣ

Παρακάτω αναφέρουμε κάποιες από τις πηγές τις οποίες συμβουλευτήκαμε κατά την διάρκεια της άσκησης, πέρα από τις οδηγίες που δόθηκαν στο εργαστήριο.

1. https://www.glfw.org/docs/3.3/input_guide.html
2. <https://www.opengl-tutorial.org/beginners-tutorials/tutorial-2-the-first-triangle/>
3. <https://stackoverflow.com/questions/10150751/opengl-drawing-a-square-with-gldrawarrays>
4. <https://stackoverflow.com/questions/7000078/what-exactly-does-glenablevertexattribarray-do>
5. <https://stackoverflow.com/questions/37951641/can-i-have-multiple-gl-array-buffer-buffers>
6. <https://stackoverflow.com/questions/15821969/what-is-the-proper-way-to-modify-opengl-vertex-buffer>
7. <https://computergraphics.stackexchange.com/questions/5712/gl-static-draw-vs-gl-dynamic-draw-vs-gl-stream-draw-does-it-matter>